Using Large Language Models Effectively

1) Why using an LLM well is different from just having an LLM

A large language model may be very powerful, but the quality of its output depends a lot on **how you use it**. The whitepaper says that **prompt engineering and sampling techniques have a strong influence on LLM performance**. Prompt engineering affects what the model understands you want, and sampling affects how it chooses the words it generates.

So even if you use the same model, you can get very different results depending on:

- how clearly you ask,
- whether you give examples,
- how much background you include,
- and which decoding parameters you choose.

Easy idea

An LLM is not just "smart by itself."
It also needs:

- the right **guidance**,
- the right **instructions**,
- and sometimes the right **generation settings**.

---

2) What prompt engineering means

The whitepaper defines **prompt engineering** as the process of designing and refining the text inputs, or prompts, that you feed into an LLM to achieve desired and relevant outputs.

In simple words, prompt engineering means:

writing your input in a way that helps the model give the kind of answer you actually want.

Why this matters

The whitepaper says LLMs are powerful, but they still need guidance to unleash their full potential. Prompt engineering can help either:

- **ground the model** toward factual and focused answers,
- or **open up creativity**, such as when you want a story or poem.

So prompt engineering is not only for accuracy.
It is also for controlling:

- style,
- tone,
- structure,
- level of detail,
- and task direction.

---

3) What makes a good prompt

The whitepaper lists common prompt engineering methods such as:

- providing clear instructions,
- giving examples,
- using keywords,
- formatting to emphasize important information,
- and adding background details.

Let's understand each one.

3.1 Clear instructions

The more clearly you say what you want, the easier it is for the model to respond well.

Weak prompt

"AI notes"

This is too vague.

Better prompt

"Give me beginner-friendly notes on supervised learning with definitions, examples, and simple explanations."

This is much clearer.

3.2 Give examples

Examples help the model understand the pattern or format you want.

Example

“Convert these informal sentences into professional email language.”

Then provide 2 or 3 examples first.

This often improves the answer quality because the model now sees the style it should imitate.

3.3 Use keywords

Keywords help anchor the model on the right topic or output style.

Example

If you want technical notes, adding words like:

- definition,
- procedure,
- formula,
- example,
- advantages,
- disadvantages

can guide the structure strongly.

3.4 Formatting matters

The whitepaper mentions formatting to emphasize important information.

That means you can improve responses by explicitly structuring the prompt.

Example

Instead of:

“Explain clustering”

You might say:

“Explain clustering in this format:

1. definition
2. how it works
3. example
4. advantages
5. disadvantages”

This makes the output more organized.

3.5 Background details

The more useful context you provide, the better the model can tailor the response.

Example

“Explain logistic regression to a beginner who knows basic statistics but not machine learning.”

This is much better than just saying:

“Explain logistic regression.”

Because now the model knows:

- audience level,
- expected depth,
- and teaching style.

---

4) Why prompt engineering works

An LLM is trained on huge amounts of text and knows many possible ways to answer. But without clear guidance, it may choose a response style that does not match your need.

Prompt engineering helps narrow the model’s choices.

Easy analogy

Suppose you ask a person:

“Tell me about Python.”

They might talk about:

- the programming language,
- the snake,
- beginner syntax,
- advanced frameworks,
- or job roles.

But if you ask:

“Teach me Python loops like I am a beginner, with examples and common mistakes,”

the response becomes much more useful.

That is exactly why prompt engineering matters.

---

5) Zero-shot prompting

The whitepaper defines **zero-shot prompting** as giving the LLM a direct prompt with instructions, without additional examples. The model relies on its existing knowledge to produce the response. This requires no extra training examples in the prompt, but may be less reliable than few-shot prompting.

Simple meaning

You just tell the model what to do.

Example

“Summarize this paragraph in simple words.”

No examples are provided. The model handles the task directly.

When zero-shot works well

Zero-shot is useful when:

- the task is common,
- the instruction is clear,
- and the model already has enough knowledge about the task.

Limitation

Because there are no examples, the model may misunderstand:

- format,
- tone,
- level of detail,
- or task expectations.

---

6) Few-shot prompting

The whitepaper defines **few-shot prompting** as giving the model a task description plus a small number of carefully chosen examples, often around three to five. These examples guide the model's response.

Why few-shot helps

Sometimes the model understands the task better when it sees a pattern first.

Example

Prompt:

Country: France → Capital: Paris
Country: Japan → Capital: Tokyo
Country: Egypt → Capital: Cairo
Country: Brazil → Capital:

The model then continues:

Brasília

This is few-shot prompting.

Why it is powerful

Few-shot prompting is especially useful when:

- formatting matters,
- the task is unusual,
- output style must be consistent,
- or zero-shot results are not reliable enough.

---

7) Zero-shot vs few-shot

This difference is important.

Zero-shot

You only give instructions.

Few-shot

You give instructions plus examples.

Easy rule

- Use **zero-shot** when the task is straightforward.
- Use **few-shot** when the model needs help understanding the exact pattern you want.

Example comparison

Zero-shot

“Classify this movie review as positive or negative.”

Few-shot

“Review: I loved the acting. → Positive
Review: The plot was boring. → Negative
Review: The visuals were stunning. → Positive
Review: The dialogue felt weak. → ?”

Few-shot often gives more controlled results.

---

8) Chain-of-thought prompting

The whitepaper explains **chain-of-thought prompting** as a technique that improves performance on complex reasoning tasks. Instead of only asking a question directly, you provide a prompt that shows how to solve similar problems step by step. Then the model generates reasoning for the new problem before giving the answer.

Simple meaning

You encourage the model to reason through the problem in smaller steps.

Why this helps

Complex tasks often fail when the model jumps too quickly to an answer.

Breaking the task into steps can improve:

- logic,
- arithmetic,
- structured problem-solving,
- and explanation quality.

Example

Instead of asking:

“If a train travels 60 miles in 2 hours, what is its speed?”

You may prompt the model with step-by-step examples of:

- identifying known values,
- using formula,
- calculating result.

This often improves reasoning consistency.

---

9) Important caution about chain-of-thought

The whitepaper presents chain-of-thought as useful for complex reasoning, but the main practical lesson is this:

Use it when:

- the task has multiple reasoning steps,
- the user wants explanation,
- or the problem is harder than simple fact recall.

For simple tasks, chain-of-thought is often unnecessary.

Example where it helps

- math word problems

- multi-step logic questions
- complex planning
- explanation of algorithms

Example where it may not be needed

- “What is the capital of Canada?”
- “Translate hello into Spanish.”

---

10) Prompt engineering as an active research area

The whitepaper explicitly says prompt engineering is an **active area of research**.

This is important because prompting is not a solved problem. Different tasks, models, and domains respond differently to:

- instruction wording,
- example ordering,
- context amount,
- and output constraints.

That is why in practice, people often test several prompt versions before settling on the best one.

Practical lesson

Do not assume your first prompt is the best prompt.

Often, better prompts come from:

- revising wording,
- tightening instructions,
- adding examples,
- and clarifying output format.

Part B: Sampling Techniques and Generation Parameters

11) What sampling means in LLMs

After the model predicts probabilities for the next token, it still needs a rule for **choosing** the next token. The whitepaper says sampling techniques determine how output tokens are chosen and influence the **correctness, creativity, and diversity** of the output.

This means two things:

- the model gives probabilities,
- but the decoding strategy decides what actually gets generated.

Easy example

Suppose the next-token probabilities are:

- “great” = 0.45
- “good” = 0.30
- “excellent” = 0.15
- “terrible” = 0.01

Different sampling strategies may choose differently.

That choice affects whether output is:

- predictable,
- varied,
- safe,
- repetitive,
- or creative.

---

12) Why sampling matters

Sampling does not change the model’s underlying knowledge.
It changes how that knowledge is expressed.

The whitepaper says these methods are essential for controlling output quality, creativity, and diversity.

Simple interpretation

A model may know many reasonable next words. Sampling decides whether it:

- always picks the most likely one,
- explores alternatives,
- or balances both.

This is why the same prompt can produce:

- very stable results,
- or very imaginative ones,
  depending on the sampling settings.

---

13) Greedy search

The whitepaper defines **greedy search** as selecting the token with the highest probability at each step. It is the simplest option, but it can lead to repetitive and predictable outputs.

How it works

At every step:

- look at all candidate next tokens,
- pick the one with the highest probability,
- repeat.

Example

If the model predicts:

- “is” = 0.60
- “was” = 0.20
- “seems” = 0.10

greedy search always picks:

“is”

Advantage

Greedy search is:

- simple,
- deterministic,

- and often stable.

Disadvantage

It can become:

- repetitive,
- dull,
- and locally optimal but not globally best.

Easy intuition

Greedy search always chooses the most obvious next word.

That can be useful for some tasks, but sometimes it makes writing feel too rigid.

---

14) Random sampling

The whitepaper says **random sampling** selects the next token according to the probability distribution, meaning each token is sampled proportionally to its predicted probability. This can create more surprising and creative text, but it also increases the chance of nonsensical output.

How it works

Instead of always choosing the top token, the model samples from the probability distribution.

That means:

- high-probability tokens are still more likely,
- but lower-probability tokens can sometimes be chosen too.

Example

If probabilities are:

- “good” = 0.50
- “great” = 0.30
- “excellent” = 0.15
- “banana” = 0.01

random sampling usually chooses “good” or “great,” but sometimes may choose something less likely.

Advantage

More creative and varied outputs.

Disadvantage

Higher risk of strange or lower-quality outputs.

---

15) Temperature sampling

The whitepaper explains **temperature sampling** as adjusting the probability distribution using a temperature parameter. Higher temperatures promote diversity, while lower temperatures favor high-probability tokens.

This is one of the most important decoding concepts.

What temperature really does

Temperature changes how “sharp” or “flat” the probability distribution is.

- **Low temperature** makes the model more confident and conservative.
- **High temperature** makes it more exploratory and diverse.

Easy intuition

Think of temperature like controlling randomness.

- low temperature = safer, more focused
- high temperature = more adventurous, more creative

---

16) Low temperature vs high temperature

Low temperature

With low temperature, the model strongly favors the top predicted tokens.

This is useful when you want:

- accuracy,
- consistency,

- and less randomness.

Good use cases

- summarization
- factual question answering
- technical explanations
- translation
- code generation

High temperature

With higher temperature, the probability distribution becomes flatter, so more token options become possible.

This is useful when you want:

- creativity,
- variation,
- brainstorming,
- or storytelling.

Good use cases

- poems
- creative writing
- alternative phrasings
- idea generation

Tradeoff

Higher temperature can also increase the chance of:

- inconsistency,
- off-topic wording,
- and nonsense.

---

17) Temperature procedure example

Suppose the model gives these next-token probabilities:

- “beautiful” = 0.50
- “wonderful” = 0.25
- “nice” = 0.15
- “strange” = 0.05

With low temperature

The distribution becomes sharper.
The model is even more likely to choose:

“beautiful”

With high temperature

The distribution becomes flatter.
Now “wonderful” or “nice” become more likely too, and even less likely options become more possible.

Result

- low temperature → more predictable
- high temperature → more diverse

---

18) Top-k sampling

The whitepaper defines **top-k sampling** as randomly sampling from the top **K** most probable tokens. The value of K controls the degree of randomness.

How it works

1. Sort tokens by probability.
2. Keep only the top K candidates.
3. Sample randomly from those K.

Example

If K = 3 and token probabilities are:

- “great” = 0.40
- “good” = 0.30
- “excellent” = 0.15
- “odd” = 0.05
- “banana” = 0.01

Then only these three are kept:

- “great”
- “good”
- “excellent”

The rest are ignored.

Why this is useful

Top-k limits randomness so the model does not sample from extremely unlikely tokens.

This often improves output quality while still allowing variation.

---

19) How K affects output

Small K

Small K means:

- fewer choices,
- less randomness,
- more controlled output.

Large K

Large K means:

- more choices,
- more variation,
- and potentially more creativity.

Tradeoff

If K is too small, output may become repetitive.

If K is too large, output may become noisy or less coherent.

So top-k is a way of balancing:

- quality,
- diversity,
- and randomness.

---

20) Top-p sampling

The whitepaper summary mentions **Top-P** as another important parameter to tune for the right balance of correctness, diversity, and creativity.

Top-p is also called **nucleus sampling**.

Main idea

Instead of taking a fixed number of tokens like top-k, top-p takes the **smallest set of tokens whose total probability reaches a threshold p**.

Example

Suppose probabilities are:

- A = 0.50
- B = 0.25
- C = 0.15
- D = 0.05
- E = 0.03
- F = 0.02

If top-p = 0.80, then keep tokens until total probability reaches 0.80:

- A + B + C = 0.90

So only A, B, and C are used.

Why top-p is useful

Unlike top-k, it adapts to the shape of the probability distribution.

- if the model is very confident, only a few tokens are kept
- if the model is less confident, more tokens may be included

That makes top-p flexible.

---

21) Top-k vs top-p

This is a very common comparison.

Top-k

Keeps a fixed number of tokens.

Top-p

Keeps a variable number of tokens based on total probability mass.

Easy way to remember

- **Top-k** asks: “How many choices should I allow?”
- **Top-p** asks: “How much probability mass should I allow?”

Practical difference

Top-p often adapts better because the number of kept tokens changes depending on how confident the model is.

---

22) Max decoding steps

The whitepaper summary also mentions **max decoding steps** as one of the parameters you can tweak to find the right mix of correctness, diversity, and creativity.

What it means

This controls how long the model is allowed to keep generating.

Why it matters

If max decoding steps is too small:

- the answer may stop too early.

If it is too large:

- the model may become repetitive,

- drift off-topic,
- or generate more than needed.

Example

For a short factual answer, small output length may be enough.

For story writing or long explanation, larger decoding length is needed.

---

23) How prompting and sampling work together

This is one of the most important practical ideas.

A good answer usually depends on both:

- a good prompt,
- and suitable sampling settings.

Example 1: factual summary

If your task is:

“Summarize this article accurately in 5 bullet points.”

You usually want:

- clear prompt,
- lower temperature,
- controlled sampling.

Because you care about correctness more than creativity.

Example 2: poem generation

If your task is:

“Write a creative poem about winter.”

You may want:

- a style-rich prompt,
- higher temperature,
- more flexible sampling.

Because creativity matters more.

---

24) Procedure example: using an LLM for a factual task

Suppose you want a reliable technical explanation.

Prompt

“Explain overfitting in machine learning for a beginner. Use simple language, one definition, one real-world example, and 3 key points.”

Recommended style

- very clear instruction
- structured output format
- some grounding context
- lower randomness settings

Why

Because your goal is:

- clarity,
- correctness,
- and controlled explanation.

---

25) Procedure example: using an LLM for a creative task

Suppose you want a story.

Prompt

“Write a short emotional story about a student who overcomes failure. Make it motivational and human.”

Recommended style

- expressive prompt
- emotion/tone specified
- higher temperature

- more flexible sampling

Why

Because here you want:

- variety,
- originality,
- and creativity.

---

26) Common mistakes in prompting

Based on the whitepaper's ideas, some common problems are:

a) Being too vague

Bad:

"Write about AI."

Too open-ended.

b) Not defining output format

If you need bullets, steps, code, summary, or short answer, say that explicitly.

c) Forgetting audience level

If the response should be beginner-friendly, technical, academic, or professional, mention that.

d) Using highly random settings for factual tasks

That may hurt correctness.

e) Expecting one prompt to work perfectly immediately

Prompt engineering often requires iteration because LLMs can be sensitive to prompts.

---

27) Practical guidelines for choosing parameters

The whitepaper's overall lesson is that you should choose prompting and sampling settings based on the task, because there is always some tradeoff between correctness, diversity, and creativity.

For factual and technical tasks

Usually prefer:

- clear instructions
- explicit format
- low temperature
- limited randomness

For structured tasks

Often use:

- few-shot examples
- format constraints
- moderate randomness if variation is acceptable

For creative tasks

Usually allow:

- richer descriptive prompts
- higher temperature
- more flexible sampling

---

28) Best overall lesson from this section

The whitepaper makes a very practical point: after the model has been trained and tuned, it is still important to **experiment with prompt engineering** and to adjust parameters like **top-k, top-p, and max decoding steps** to find the right balance for the task.

So the best way to think about LLM use is this:

1. Decide what kind of task you have.
2. Write a prompt that clearly communicates that task.
3. Add examples if necessary.
4. Choose generation settings based on whether you want:
    - accuracy,

    - consistency,
    - diversity,
    - or creativity.
5. Test and refine.

That is the practical workflow.

1. Step-Back Prompting

Instead of asking a model to solve a specific, potentially narrow problem immediately, you "step back" to ask a broader, more fundamental question.

- **The Concept**: You prompt the LLM to identify the underlying principles or general concepts first.
- **The Benefit**: This activates the model's deep background knowledge, reducing errors on specific details and mitigating biases.
- **Example**: If asking about a specific level in a game, first ask: "What are the 5 key settings that make a first-person shooter level engaging?" Then use those settings as context for the final task.

2. Tree of Thoughts (ToT)

While Chain of Thought (CoT) is a single linear path of reasoning, **Tree of Thoughts** allows the model to explore multiple branches at once.

- **The Concept**: The model maintains a "tree" where each node is an intermediate "thought" or step.
- **The Action**: The model can branch out, evaluate different reasoning paths, and even "look ahead" or backtrack to find the best global solution.

3. Self-Consistency

This is a "democracy" for LLM reasoning.

- **The Concept**: You run the same CoT prompt multiple times (using a high temperature to ensure variety).
- **The Process**: You extract the answer from each path and perform a **majority vote**.

- **The Benefit**: If 4 out of 5 reasoning paths lead to "IMPORTANT" and only one says "NOT IMPORTANT," the model is more likely to be correct by choosing the most consistent result.

4. ReAct (Reason & Act)

This is the bridge between a "chatbot" and an **"AI Agent"**.

- **The Concept**: It combines natural language reasoning with the ability to use **external tools** (like a search engine or calculator).
- **The Loop**: The model follows a "Thought -> Action -> Observation" cycle.
    - **Thought**: "I need to find the population of France."
    - **Action**: Search[population of France]
    - **Observation**: "67 million."
- **Implementation**: This is typically done using frameworks like **LangChain** to connect the LLM to APIs.

What is the KV Cache?

When an LLM generates text, it does so **autoregressively** (one token at a time). To predict the *next* token, the model needs to "look back" at all previous tokens.

Without a cache, the model would have to re-calculate the mathematical representations (the **Key** and **Value** vectors) for every single previous word, every single time it generates a new one. This causes computational costs to grow quadratically, making long conversations impossibly slow.

The Solution: The **KV Cache** stores these vectors in GPU memory after they are calculated once. For subsequent tokens, the model simply "fetches" the old data and only calculates the math for the one brand-new token.

Latency Issues & The "Memory Wall"

While the KV Cache solves the *computation* bottleneck, it creates a new **memory and latency bottleneck**:

1. **VRAM Consumption:** The cache grows linearly with the length of the conversation and the number of users. For a large model like Llama 3-70B, a single user's 4,000-token conversation can take up over **10 GB of GPU memory**.
2. **Time to First Token (TTFT):** This is the delay a user feels after hitting "enter". During this phase (called **Prefill**), the model must process your entire prompt to build the initial KV Cache. If your prompt is a long document, the TTFT can be several seconds.
3. **Memory Bandwidth Bottleneck:** During the generation phase (called **Decode**), the model spends more time moving data from memory to the processor than it does actually performing calculations. This is why even powerful GPUs can sometimes feel slow when generating long responses.

How Engineers Fix These Issues

In your data engineering work, you might encounter these optimization strategies:

- **PagedAttention:** Much like virtual memory in an OS, this breaks the KV Cache into small blocks to prevent memory fragmentation.
- **Quantization:** Reducing the precision of the cache (e.g., from 16-bit to 8-bit or 4-bit) to cut its memory footprint in half, though this can occasionally lead to "fuzzy" reasoning or hallucinations in very long contexts.
- **Offloading:** Moving the cache from the expensive, limited GPU memory (HBM) to slower but cheaper System RAM or SSDs when it's not immediately needed.